SYSTEM DESIGN FOR ML

Ayrintili Konu Anlatimi

Modul 48 - Veri Bilimci Mufredati 2026

Bu modulu bitirdiginde
Bir ML modelini sadece notebook icinde egitmek yerine; verinin sisteme gelisinden ozellik uretimine, egitimden deployment ve monitoring asamasina kadar production seviyesinde bir mimariyi tasarlayabilir, sistem tasarimi mulakatinda trade-off tartisabilir hale gelmen hedeflenir.

Kapsam: batch ve online inference, feature pipeline, feature store, training pipeline, model serving, olceklenebilirlik, deployment stratejileri, guvenilirlik, monitoring, drift, retraining, guvenlik, maliyet, gercek sistem tasarimi vakasi ve mulakat kontrol listesi.

Icindekiler

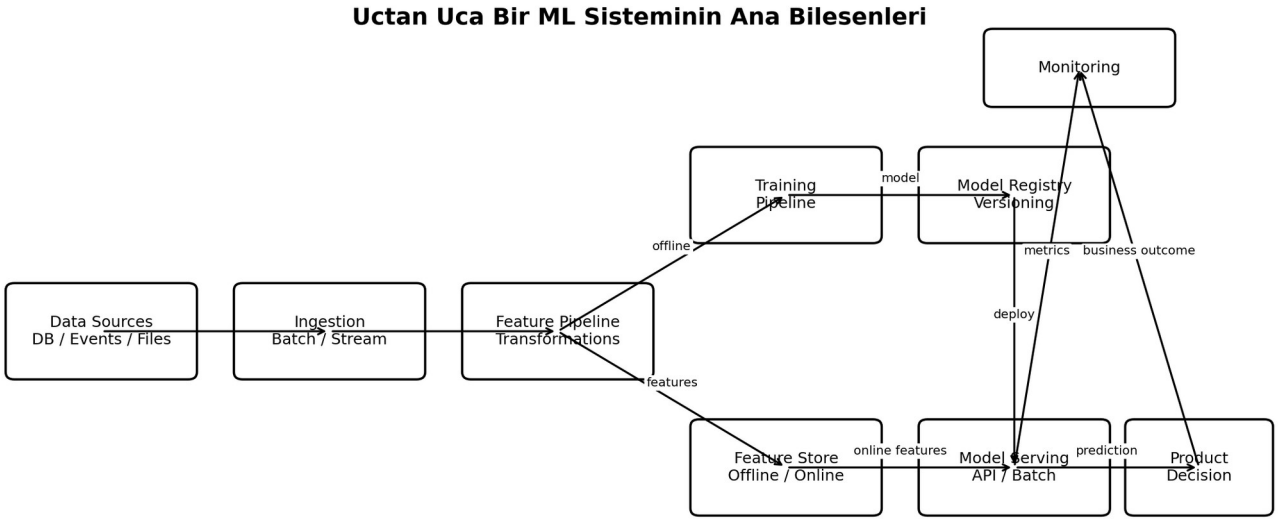
1. 1. ML System Design nedir?
2. 2. Gereksinim analizi: once problemi tanimla
3. 3. Uctan uca ML sistem mimarisi
4. 4. Veri katmani ve ingestion
5. 5. Feature engineering pipeline ve feature store
6. 6. Training pipeline tasarimi
7. 7. Batch inference
8. 8. Online / real-time inference
9. 9. Model serving tasarimi
10. 10. Olceklenebilirlik ve performans
11. 11. Deployment stratejileri
12. 12. Reliability ve failure handling
13. 13. ML monitoring
14. 14. Drift ve retraining
15. 15. Versioning ve reproducibility
16. 16. Security, privacy ve governance
17. 17. Maliyet optimizasyonu
18. 18. Yaygin ML sistem desenleri
19. 19. Uctan uca vaka: fraud detection sistemi
20. 20. ML system design mulakati nasil cozulur?
21. 21. Alistirmalar ve tasarim sorulari
22. 22. Terimler sozlugu ve tekrar kontrol listesi

Onemli bakis acisi
System design sorularinda tek bir 'dogru mimari' yoktur. Degerlendirilen sey; gereksinimlerden mantikli varsayimlar turetebilmen, alternatifler arasindaki trade-offlari aciklaman, darboğazlari ongorebilmen ve tasarimini olculebilir SLO/KPI'larla savunabilmendir.

1. ML System Design Nedir?

ML system design, bir makine ogrenmesi modelinin gercek bir urun veya operasyon icinde guvenilir, hizli, olceklenebilir ve izlenebilir bicimde calismasi icin gereken tum sistemi tasarlama disiplinidir.

Klasik bir veri bilimi projesinde “veriyi al - modeli egit - metriği raporla” yeterli olabilir. Production sisteminde ise model, daha buyuk bir dagitik sistemin sadece bir bilesenidir. Veri gecikirse, feature yanlis hesaplanirsa, API cok yavaslarsa veya prediction dagilimi degisirse modelin offline AUC degeri ne kadar iyi olursa olsun sistem basarisiz olabilir.



Sekil 1 - Uctan uca ML sisteminde veri, feature, model, serving ve monitoring birbirine baglidir.

1.1 Model performansi ile sistem performansi ayni sey degildir

Boyut	Model odakli soru	Sistem odakli soru
Dogruluk	AUC / RMSE kac?	Dogruluk gercek trafikte de korunuyor mu?
Gecikme	Yok	p95/p99 latency hedefi nedir?
Kapasite	Yok	Saniyede kac prediction desteklenmeli?
Veri	Dataset hazir	Feature ne zaman ve nasil guncelleniyor?
Dayaniklilik	Yok	Servis veya feature store cokerse ne olur?
Degisim	Model sabit	Drift olursa nasil tespit edilir ve retrain edilir?
Yonetim	model.pkl	Hangi veri + kod + feature + model versiyonu productionda?

Ana fikir
Production ML: Model + Data + Software + Infrastructure + Operations. Bunlardan herhangi biri zayifsa tum sistem zayiflar.

2. Gereksinim Analizi: Once Problemi Tanimla

Sistem tasarimina teknoloji secerek baslanmaz. Once urun ihtiyaci ve sistem gereksinimleri netlestirilir. Bir mulakatta ilk 5-10 dakika burada harcanabilir.

2.1 Fonksiyonel gereksinimler

- **Prediction ne uretilecek?** Sinif, skor, siralama, forecast, embedding, oneriler vb.
- **Prediction ne zaman gerekli?** Kullanici istegi aninda mi, saatlik mi, gunluk mu?
- **Girdi nedir?** Kullanici ozellikleri, transaction, text, image, event stream?
- **Cikti nereye gider?** API response, veri tabani, queue, dashboard, risk motoru?
- **Feedback/label nasil gelir?** Anlik mi, saatler/gunler sonra mi?

2.2 Non-functional requirements

Gereksinim	Ornek soru	Neden onemli?
Latency	Prediction 50 ms altinda mi olmali?	Mimariyi ve feature erisimini belirler.
Throughput	1K mi 100K request/s mi?	Replica, cache ve serving altyapisini belirler.
Availability	99.9% mi 99.99% mi?	Failover ve redundancy gereksinimini etkiler.
Freshness	Feature 1 dk mi 24 saat mi eski olabilir?	Batch vs stream kararini etkiler.
Consistency	Her yerde ayni feature tanimi mi?	Training-serving skew riskini azaltir.
Cost	Prediction basina maliyet limiti?	Model boyutu ve altyapi secimini etkiler.
Privacy	PII var mi?	Sifreleme, erisim ve retention politikasini etkiler.

2.3 Hizli kapasite tahmini

Sistem tasariminda kabaca kapasite hesaplamak mimari secimlerini somutlastirir. Ornegin 20 milyon aktif kullanicisi olan bir urunde, kullanici basina gunde ortalama 8 prediction istiyorsak:

```
20,000,000 users x 8 predictions/day = 160,000,000 predictions/day
Average QPS ~= 160,000,000 / 86,400 ~= 1,852 QPS
Peak factor 5x => ~9,260 QPS peak
```

Bu hesap kesin kapasite plani degildir; ama tek bir instance mi yoksa yatay olceklenen bir serving cluster mi gerekecegini tartismaya yarar.

3. Uctan Uca ML Sistem Mimarisi

Genel bir production ML sistemi iki dongu halinde dusunulebilir: offline learning loop ve online serving loop.

3.1 Offline learning loop

23. Ham veriyi veri ambarina/lake'e al.
24. Featurelari point-in-time dogru sekilde uret.
25. Train/validation/test datasetlerini olustur.
26. Modeli egit, degerlendir ve deneyleri kaydet.
27. Basarili modeli registryye versiyonlu kaydet.
28. Deployment politikasina gore productiona terfi ettir.

3.2 Online serving loop

29. Client veya upstream servis prediction istegi gonderir.
30. Online featurelar dusuk gecikmeyle okunur.
31. Model inference calisir.
32. Prediction + model version + feature metadata loglanir.
33. Prediction urun kararina donusur.
34. Sonradan gelen gercek label ile model performansi olculur.

Critical path
Online inference yolunda bulunan her network hop, feature lookup ve model computation latency butcesinden yer. Bu nedenle online yol olabildigince kisa ve deterministic tutulur.

4. Veri Katmani ve Ingestion

4.1 Veri kaynaklari

- Transactional database (OLTP)
- Data warehouse / lakehouse
- Event stream
- Third-party API
- Logs ve telemetry
- Dosya/object storage

4.2 Batch ingestion

Batch ingestion, veriyi belirli araliklarla tasir. Gunluk churn skoru, haftalik demand forecast veya gece hesaplanan segmentler icin genellikle yeterlidir.

Avantaj	Dezavantaj
Basit operasyon, tekrar calistirma kolay	Feature freshness dusuk olabilir
Buyuk hacimlerde verimli	Schedule ve dependency yonetimi gerekir
Maliyet kontrolu kolay	Anlik risk/oneriler icin uygun degil

4.3 Stream ingestion

Stream sistemlerinde eventler olustukca islenir. Fraud detection, real-time personalization veya operasyonel alert gibi senaryolarda kullanilir.

- **Event time vs processing time:** Eventin gercekte oldugu an ile sistemin isledigi an farklidir.
- **Late events:** Gec gelen eventler icin watermark/window stratejisi gerekir.
- **Exactly-once miti:** Bir cok sistem gercekte at-least-once delivery + idempotent processing ile guvenilirlik saglar.
- **Ordering:** Global siralama pahali olabilir; partition/key bazinda siralama daha yaygindir.

4.4 Veri kalitesi kontratlari

```
Example checks:
- user_id NOT NULL
- amount >= 0
- event_time <= now + tolerance
- country in allowed_values
- row_count deviation < 30%
```

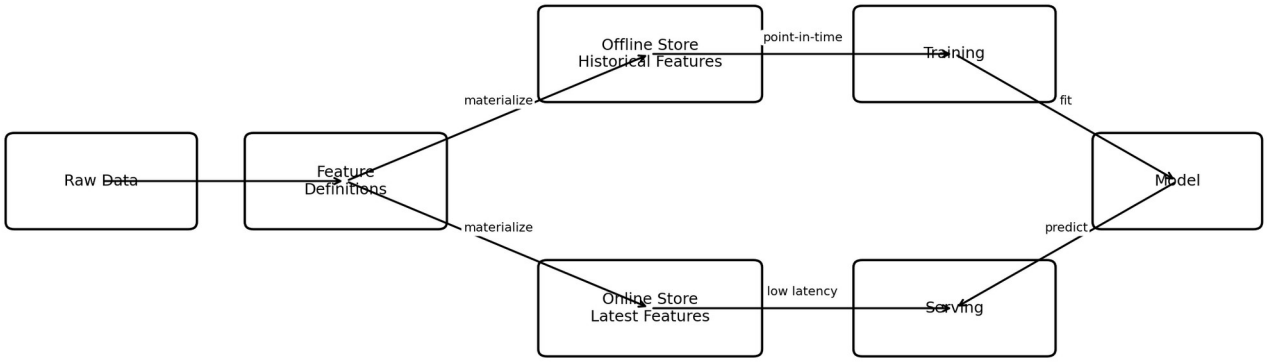
```
- duplicate transaction_id == 0
```

Model monitoringden once data monitoring gerekir. Modelin girdisi bozuksa prediction kalitesi de bozulur.

5. Feature Engineering Pipeline ve Feature Store

Feature pipeline, raw veriyi modele uygun ve tekrar kullanilabilir ozelliklere donusturur. Productiondaki temel problem, training sirasinda hesaplanan feature ile serving sirasinda hesaplanan featurein farkli olabilmesidir. Buna training-serving skew denir.

Feature Store: Training-Serving Tutarliligi



Sekil 2 - Offline ve online feature erisimi ayni feature tanimlarindan beslenmelidir.

5.1 Feature store ne cozer?

- Feature tanimlarini merkezi ve tekrar kullanilabilir hale getirir.
- Offline historical feature ile online latest feature erisimini ayirir.
- Point-in-time correct training dataset olusturmayi kolaylastirir.
- Feature ownership, metadata ve lineage saglayabilir.
- Ayni featurein farkli ekiplerce farkli hesaplanmasini azaltir.

5.2 Point-in-time correctness

Bir prediction t aninda yapildiysa training datasetine sadece t aninda bilinebilen bilgiler girmelidir. Gelecekten gelen bilgi modele sizarsa data leakage olur.

```
YANLIS:
transaction at 10:00 -> feature = customer_daily_total computed at 23:59

DOGRU:
transaction at 10:00 -> feature = customer_total_until_10_00
```

5.3 Feature freshness siniflari

Feature	Freshness	Uygun yontem

Yas / ulke	Haftalar-aylar	DB / cache
Son 30 gun harcama	Saatler	Batch materialization
Son 5 dk transaction sayisi	Saniyeler	Streaming + online store
Current device risk	Milisaniye-saniye	Request-time computation / cache

Trade-off
Her featurei real-time yapmak gereksiz maliyet ve karmasiklik yaratir. Feature freshness sadece urun ihtiyaci kadar iyi olmalidir.

6. Training Pipeline Tasarimi

Training pipeline, model egitimini manuel notebook adimindan cikartip tekrarlanabilir bir is akisi haline getirir.

35. Data snapshot / dataset version belirle.
36. Data validation calistir.
37. Feature dataset olustur.
38. Train-validation-test split yap.
39. Model training calistir.
40. Offline evaluation ve slice metrics hesapla.
41. Artifactlari kaydet: model, preprocessor, config, metrics.
42. Promotion gate uygulayarak registryye kaydet.

6.1 Promotion gate ornegi

```
if auc >= 0.87 and recall_fraud >= 0.75 and latency_ms <= 20:
    register_model(candidate)
else:
    fail_pipeline("Model does not satisfy release criteria")
```

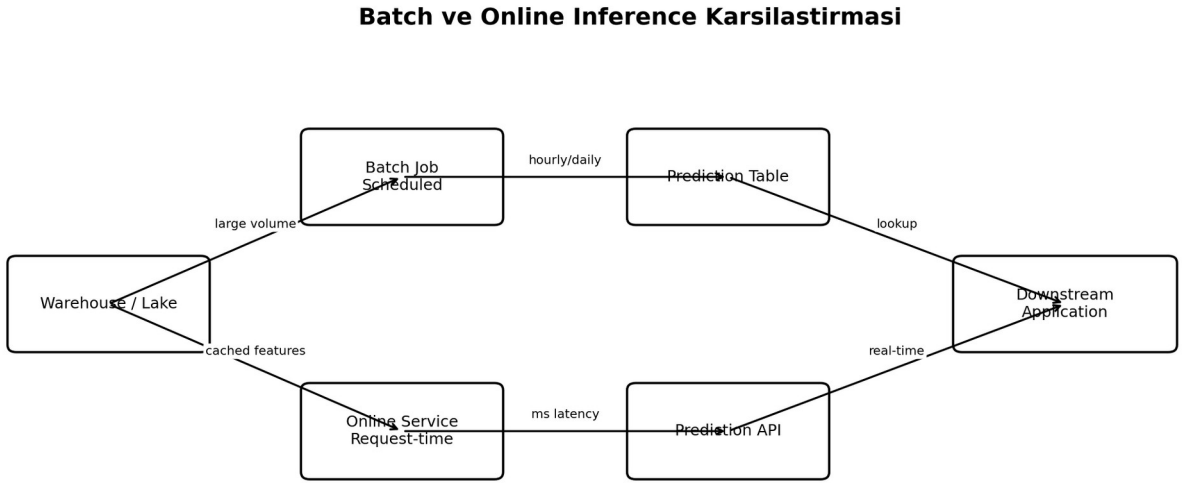
Gercek sistemlerde tek bir ortalama metric yeterli olmayabilir. Segment bazli performans, calibration, fairness, inference latency ve model boyutu da release gate icinde olabilir.

6.2 Scheduled vs event-driven retraining

Yontem	Ne zaman?	Risk
Scheduled	Haftalik/aylik veri degisimi	Gereksiz retraining olabilir
Drift-triggered	Dagilim anlamli degisince	False alarm / gecikmeli label problemi
Performance-triggered	Gercek kalite dusunce	Label gelmesi gecikebilir
Manual approval	Regule/yuksek riskli sistem	Operasyonel yavaslik

7. Batch Inference

Batch inference, cok sayida ornegin belirli araliklarla toplu skorlanmasidir. Sonuclar genellikle tablo/object storage icine yazilir ve downstream sistemler buradan okur.



Sekil 3 - Batch ve online inference ayni model amacina hizmet etse de operasyonel tasarimlari farklidir.

7.1 Batch inference ne zaman iyi secim?

- Prediction anlik olmak zorunda degilse.
- Tum population periyodik skorlanacaksa.
- Yuksek throughput, dusuk birim maliyet isteniyorsa.
- Featurelar zaten warehouse icindeyse.
- Model pahali ama toplu compute verimli ise.

7.2 Batch mimarisinde dikkat edilecekler

- **Idempotency:** Ayni job tekrar calistiginda cift kayit uretmemeli.
- **Partitioning:** Tarih/musteri bolumu gibi mantikli partitionlar kullan.
- **Backfill:** Gecmis gunleri tekrar hesaplayabilmelisin.
- **Atomic publish:** Yarim tamamlanmis prediction datasetini tuketicilere acma.
- **Data/model version:** Her output kaydinda model versiyonu izlenebilir olmali.

8. Online / Real-Time Inference

Online inference, request geldigi anda prediction uretir. Kullaniciya gosterilecek recommendation, fraud blocking, dynamic pricing veya search ranking gibi senaryolarda kritik olabilir.

8.1 Latency budget dusuncesi

```
Target p95 end-to-end latency = 100 ms
- Network + gateway:       15 ms
- Feature fetch:           25 ms
- Preprocessing:            5 ms
- Model inference:         20 ms
- Business rules:          10 ms
- Safety margin:           25 ms
```

Bu butce tasarimi, hangi featurelarin precompute edilmesi gerektigini ve model boyutunun ne kadar olabilecegini belirler.

8.2 Online serving icin kritik kavramlar

- **p95/p99 latency:** Ortalama latency tek basina yeterli degildir; tail latency kullanici deneyimini bozar.
- **Timeout:** Upstream servis sonsuza kadar beklememeli.
- **Fallback:** Model servisi yoksa default kural veya cached prediction kullanilabilir.
- **Caching:** Ayni kullanici/urun icin kisa sureli tekrar predictionlarda maliyeti azaltir.
- **Concurrency:** Model runtime ayni anda kac request isleyebilir?

9. Model Serving Tasarimi

9.1 Model servisleme yaklasimlari

Yaklasim	Aciklama	Uygun senaryo
Model application icinde	Model library olarak ayni process	Basit, tek servis, dusuk operasyon
Dedicated model microservice	REST/gRPC prediction servisi	Birden fazla tuketici, bagimsiz olcekleme
Model serving platform	Standart deploy/scale/metrics	Cok model ve cok ekip
Serverless inference	Talebe gore instance	Seyrek trafik, cold-start kabul edilebilir

9.2 REST mi gRPC mi?

REST	gRPC
Kolay debug, yaygin ekosistem	Daha kompakt binary protocol
JSON overhead	Dusuk latency / yuksek throughput icin avantaj
Browser ve public API uyumu yuksek	Strict schema / generated clients
Basit entegrasyon	Internal service-to-service icin guclu

9.3 API contract ornegi

```
POST /v1/fraud-score
{
  "transaction_id": "tx_123",
  "user_id": "u_987",
  "amount": 249.90,
  "merchant_id": "m_44"
}

Response:
{
  "risk_score": 0.93,
  "decision": "review",
  "model_version": "fraud_xgb_42"
}
```

Versiyonlama
API schema versiyonu ile model versiyonunu ayni sey sanma. API v1 icinde birden fazla model versiyonu deployment surecinde calisabilir.

10. Olceklenebilirlik ve Performans

10.1 Vertical vs horizontal scaling

Vertical scaling tek makineyi buyutur; horizontal scaling daha fazla replica ekler. Online prediction servislerinde stateless tasarim horizontal scalingi kolaylastirir.

- **Stateless serving:** Requestler arasinda local state tutmamak replica eklemeyi kolaylastirir.
- **Load balancing:** Trafik saglikli replicalara dagitilir.
- **Autoscaling:** CPU/GPU, queue depth, concurrency veya latency sinyaline gore replica sayisi degisir.
- **Model warmup:** Yeni instance ilk predictionda compilation/model load maliyeti yasayabilir.
- **Batching:** Birden fazla istegi birlikte infer etmek GPU verimliligini artirabilir fakat latency ekler.

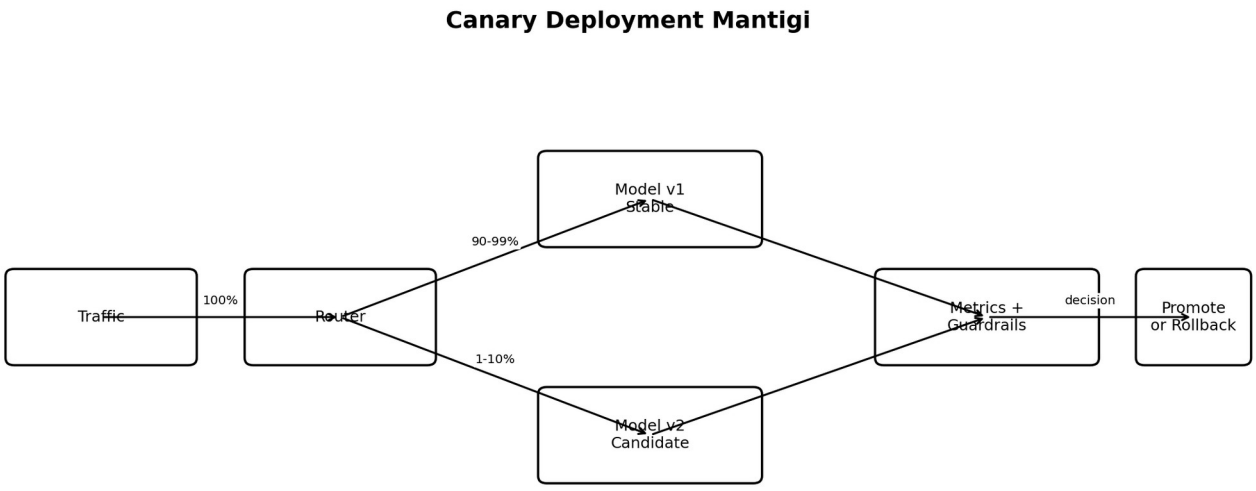
10.2 Darbogaz nerede?

Darbogaz	Belirti	Cozum ornekleri
Feature store	Latency artiyor	Cache, denormalization, read replica
Model compute	CPU/GPU doygun	Quantization, smaller model, batching, more replicas
Network	Payload buyuk	Compact schema, colocate services
Serialization	CPU overhead	Binary protocol, daha az feature
Cold start	Spike sirasinda timeout	Min replicas, pre-warming
Downstream DB	Prediction log write yavas	Async queue / buffered writes

10.3 Kuyruk tabanli async inference

Prediction sonucu aninda gerekmiyorsa request -> queue -> worker -> result store deseni, trafik patlamalarini yumusatir ve backpressure saglar. Ancak kullaniciya anlik cevap gereken path icin uygun degildir.

11. Deployment Stratejileri



Sekil 4 - Canary deploymentta yeni model once trafiğin kucuk bir yuzdesinde denenir.

Strateji	Nasil calisir?	Avantaj	Risk
Blue/Green	Eski ve yeni ortam paralel; trafik bir anda degisir	Hizli rollback	Iki ortam maliyeti
Canary	Yeni modele az trafik verilir, sonra artirilir	Risk kontrollu	Routing/metric karmasikligi
Shadow	Yeni model ayni requesti gorur ama karar vermez	Guvenli production testi	Iki kez compute
A/B test	Kullanici gruplari farkli model/strateji gorur	Business impact olcumu	Deney tasarimi gerekir

11.1 Shadow deployment neden cok degerlidir?

Offline dataset production trafik karakterini tam temsil etmeyebilir. Shadow modda aday model gercek requestleri gorur, latency ve prediction dagilimi olculur; fakat sonucu kullaniciya uygulanmaz.

11.2 Rollback kosullari

- Error rate threshold asildi.
- p99 latency SLO disina cikti.
- Business guardrail bozuldu.
- Prediction distribution beklenmedik sekilde degisti.
- Critical slice performansi bozuldu.

12. Reliability ve Failure Handling

ML servisi de diger production servisleri gibi hata verebilir. Farki, “basarili HTTP 200” donse bile prediction semantik olarak bozuk olabilir. Bu nedenle reliability hem yazilim hem veri/model katmaninda ele alinmalidir.

12.1 Temel patternler

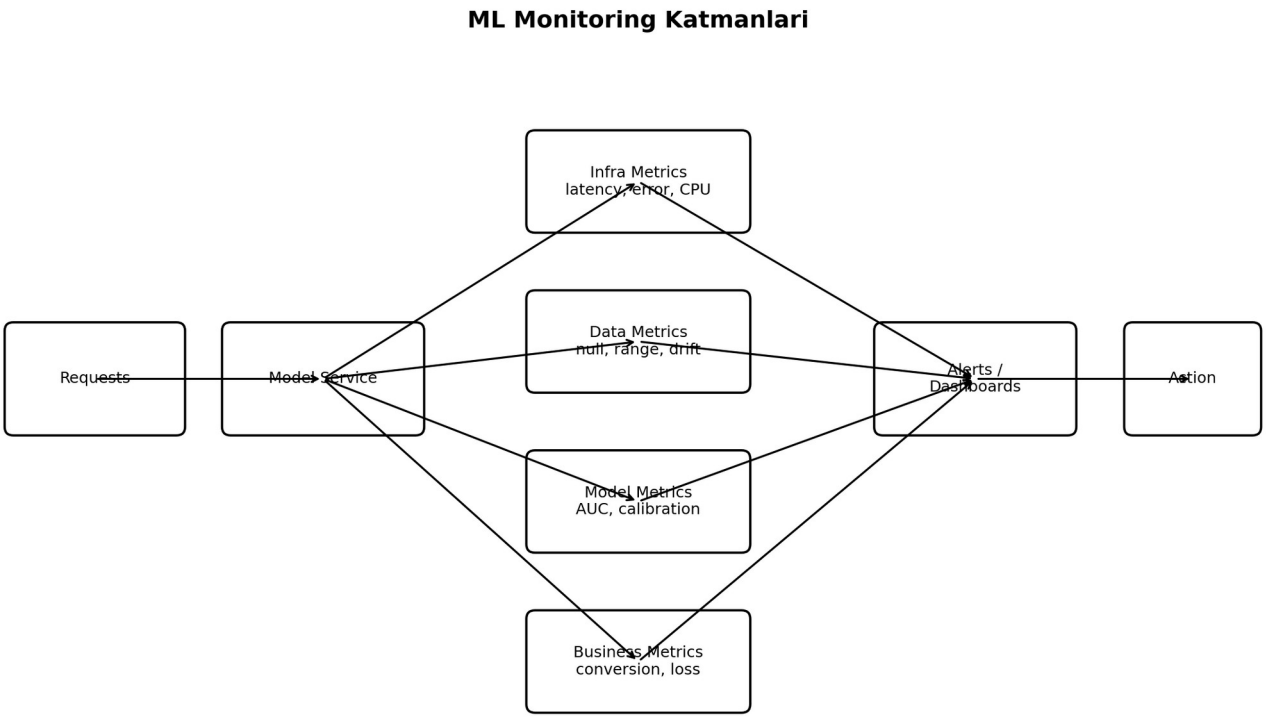
- **Timeout:** Bir dependency belli sureyi asinca beklemeyi kes.
- **Retry:** Sadece transient hatalarda ve exponential backoff ile; idempotencyyi koru.
- **Circuit breaker:** Surekli hata veren servise request gondermeyi gecici kes.
- **Bulkhead:** Bir musterinin/endpointin yukunun tum servisi cokertmesini engelle.
- **Fallback:** Cached score, heuristic rule, previous model veya safe default kullan.
- **Graceful degradation:** Tum featurelar yoksa daha basit feature setli fallback model calistir.

12.2 Availability hesabi

```
99.9% monthly availability ~= 43.8 min downtime/month
99.99% ~= 4.38 min/month
```

Yuksek availability hedefi redundancy, multi-zone deployment ve otomatik failover maliyetini artirir. Hedef, urun riskine gore belirlenmelidir.

13. ML Monitoring



Sekil 5 - Monitoring sadece CPU ve latency degildir; veri, model ve business katmanlari birlikte izlenir.

13.1 Dört monitoring katmani

Katman	Olculer	Ornek alarm
Infrastructure	CPU, GPU, memory, latency, QPS, error rate	p99 > 100 ms
Data	null, range, category, freshness, schema	null rate > %2
Model	AUC, precision, recall, calibration, score distribution	fraud recall < %70
Business	conversion, fraud loss, approval rate, revenue	approval rate -%15

13.2 Label gecikmesi problemi

Bir fraud transactionin gercekten fraudulent oldugu gunler sonra belli olabilir. Bu durumda gercek model metriğini anlik hesaplayamazsin. Proxy metrikler kullanilir: score distribution, feature drift, review rate, model disagreement, input anomaly oranlari.

13.3 Slice monitoring

Global metric stabil kalirken belirli bir ulke, cihaz, yeni kullanici grubu veya merchant segmentinde performans cokebilir. Bu nedenle kritik segmentlerde metrikleri ayri izlemek gerekir.

14. Drift ve Retraining

14.1 Data drift, concept drift, label drift

Tur	Anlam	Ornek
Data drift	P(X) degisir	Ortalama transaction amount artar
Label drift	P(Y) degisir	Fraud orani %1den %3e cikar
Concept drift	P(Y|X) degisir	Ayni davranisin fraud olasiligi degisir

Data drift her zaman model performansi dustu demek degildir; sadece input dagilimi degismistir. Bu nedenle drift alarmi otomatik retrainingin tek kosulu olmamali.

14.2 Drift olcumleri

- Population Stability Index (PSI)
- Kolmogorov-Smirnov testi
- Jensen-Shannon / KL divergence
- Categorical distribution distance
- Embedding distance / model-based drift detection

14.3 Retraining politikasini tasarlama

```
Trigger retraining if:
  enough_new_labels AND
  (performance_drop > threshold OR significant_drift) AND
  cooldown_period_passed
```

Uyari
Otomatik retraining pipelinei otomatik production promotion demek degildir. Yuksek riskli sistemlerde aday model evaluation ve approval gateinden gecmelidir.

15. Versioning ve Reproducibility

Bir prediction sorununu debug edebilmek icin “hangi model?” sorusundan fazlasina cevap vermen gerekir.

Versiyonlanmasi gereken	Ornek
Kod	git commit SHA
Dataset	snapshot/date/table version
Feature definitions	feature repo version
Config	hyperparameters / thresholds
Environment	container image / dependencies
Model artifact	model registry version
Schema	input/output contract version

15.1 Prediction lineage

```
prediction_id: p_123
request_time: 2026-09-29T09:30:12Z
```

```
model_version: fraud_xgb_42
feature_view_version: fraud_features_v7
request_schema: v1
score: 0.93
decision: review
```

Bu metadata incident incelemesinde belirli bir predictioni geriye dogru yeniden kurabilmeyi kolaylastirir.

16. Security, Privacy ve Governance

16.1 Tehdit yuzeyi

- Yetkisiz prediction API erisimi
- PII sizintisi
- Model artifact hirsizligi
- Data poisoning
- Adversarial input
- Prompt injection (LLM tabanli sistemlerde)
- Supply-chain / dependency riski

16.2 Pratik kontroller

- **Encryption:** Data at rest ve in transit sifreli.
- **IAM / least privilege:** Servis sadece ihtiyaci olan data ve modele erisebilir.
- **Secrets management:** API keyleri kod veya image icinde tutulmaz.
- **Audit logs:** Model promotion, config degisimi, data access kayit altinda.
- **PII minimization:** Modele gerekmeyen hassas veriyi hic alma.
- **Retention policy:** Prediction loglari sonsuza kadar tutulmaz.

16.3 Human-in-the-loop

Kredi, fraud, saglik, guvenlik veya yuksek maliyetli operasyonlarda model skoru dogrudan otomatik karar yerine insan incelemesine yonlendiren bir sinyal olabilir. Sistem tasarimi karar riskine gore yapilir.

17. Maliyet Optimizasyonu

ML system design sadece latency ve accuracy optimizasyonu degildir. Prediction basina compute, feature serving, storage, network ve observability maliyetleri toplam sahip olma maliyetini olusturur.

Teknik	Maliyet etkisi	Trade-off
Batch inference	Birim compute dusurur	Freshness azalir
Caching	Tekrar compute azalir	Stale prediction riski
Smaller model	CPU ve latency azalir	Accuracy kaybi olabilir
Quantization	Model compute/memory azalir	Calibration/accuracy kontrolu gerekir
Autoscaling	Idle capacity azalir	Cold start riski
Feature pruning	Store + network azalir	Model performansi etkilenebilir

Optimization sirasi
Once sistemin gercek bottleneckini olc. Profiling olmadan yapilan optimizasyon, gereksiz karmasiklik yaratabilir.

18. Yaygin ML Sistem Desenleri

18.1 Churn scoring

Tipik olarak gunluk/haftalik batch prediction yeterlidir. Data warehouse -> feature job -> batch model -> CRM prediction table. Online serving gereksiz olabilir.

18.2 Recommendation

Genellikle iki asamali mimari kullanilir: candidate generation ile binlerce adaydan yuzlerce urun secilir; ranking modeli ile son siralama yapilir. User/item embeddings precompute edilebilir, request-time context online eklenebilir.

18.3 Search ranking

Cok dusuk latency gerektirir. Retrieval -> lightweight ranker -> optional reranker deseni yaygindir. Featurelarin bir bolumu query-time hesaplanirken pahali featurelar precompute edilir.

18.4 Fraud detection

Transaction aninda skor gerekir. Streaming velocity featurelari, online store, dusuk latency model ve rule engine birlikte kullanilir. False positive maliyeti yuksek oldugu icin threshold ve human review onemlidir.

18.5 Forecasting

Cogu senaryoda batch training + batch inference uygundur. Time-based split, backtesting, seasonality, exogenous variables ve hierarchical aggregation on plana cikar.

Use case	Inference	Feature freshness	Tipik SLO
Churn	Batch	Gunluk	Saatler
Fraud	Online	Saniyeler	<100 ms
Recommendation	Online/hybrid	Dakika-saat + session	<100-200 ms
Forecast	Batch	Gunluk/haftalik	Dakikalar-saatler
Search ranking	Online	Saniye-dakika	<50-100 ms

19. Uctan Uca Vaka: Gercek Zamanli Fraud Detection Sistemi

Problem: Bir odeme platformunda her transaction icin risk skoru uretilecek. Yuksek riskli islemler reddedilecek veya manuel incelemeye gidecek. Hedef hem fraud kaybini azaltmak hem de gercek musterileri gereksiz reddetmemektir.

19.1 Gereksinimleri netlestir

Baslik	Varsayim
Traffic	Ortalama 2K QPS, peak 10K QPS
Latency	p95 < 80 ms, p99 < 150 ms
Availability	99.99% prediction endpoint
Label delay	Chargeback/confirmed fraud 1-30 gun sonra gelebilir
Decision	approve / review / decline
Fresh features	Son 5 dk ve 1 saat velocity featurelari kritik

19.2 Data ve feature tasarimi

- Historical transactions warehouse icinde tutulur.
- Real-time transaction eventleri stream platformuna yazilir.
- Stream processor user_id/card_id/device_id bazinda rolling count/sum uretir.
- Online feature store son velocity featurelarini tutar.
- Static profile featurelari daha seyrek batch materialize edilir.

19.3 Prediction path

43. Payment service transaction requestini risk servisine yollar.
44. Risk servisi request schema validation yapar.
45. Online store'dan gerekli velocity/profile featurelarini paralel okur.
46. Preprocessor feature vector olusturur.
47. Fraud model risk_score uretir.
48. Rule engine model skorunu hard rules ile birlestirir.
49. Decision payment service'e doner.
50. Request, feature metadata, score ve model version async loglanir.

19.4 Threshold ve business policy

```
score < 0.40        -> APPROVE
0.40 <= score < .80 -> REVIEW
score >= 0.80       -> DECLINE

Note: thresholds are business decisions optimized with fraud loss,
false-positive cost and review capacity.
```

19.5 Failure senaryolari

Failure	Davranis
Online feature store timeout	Cached/stale-safe features + fallback model veya rule
Model service unavailable	Previous stable model replica / safe rule path
Stream processor gecikiyor	Feature freshness alarm; velocity feature stale flag
Prediction log write fail	Decision pathini bloklama; async buffer/queue
Candidate model bozuk	Canary metric guardrail -> otomatik rollback

19.6 Monitoring dashboard

- QPS, p50/p95/p99 latency, timeout ve error rate
- Feature freshness ve null oranlari
- Risk score distribution
- Approve/review/decline oranlari
- Segment bazli false positive / recall
- Confirmed fraud loss
- Model version bazli comparison
- Stream lag ve online store latency

19.7 Retraining

Yeni label yeterince biriktiginda training pipeline yeni snapshot ile calisir. Candidate model offline evaluation, temporal backtest ve kritik segment kontrollerinden gecer. Sonra shadow/canary deployment ile production trafik karakteri test edilir. Guardraillar normalse model kademeli terfi eder.

Mülakat sonucu
Bu vaka anlatiminda yalnızca “XGBoost kullanirim” demek zayiftir. Guclu cevap; latency, freshness, stream featurelari, fallback, model registry, canary, delayed labels ve business threshold trade-offlarini birlikte aciklar.

20. ML System Design Mulakati Nasil Cozulur?

20.1 8 adimli cevap iskeleti

51. Problemi ve prediction hedefini tanimla.
52. Functional + non-functional requirements sor.
53. Traffic, data volume, latency icin kaba kapasite tahmini yap.
54. High-level architecture ciz.
55. Data ve feature pipelinei acikla.
56. Training + evaluation + registry akisini acikla.
57. Serving, scaling, deployment ve failure handlingi acikla.
58. Monitoring, drift, retraining, security ve cost ile bitir.

20.2 Sorulmasi gereken sorular

- Prediction ne kadar hizli donmeli?
- Peak QPS nedir?
- Prediction batch olabilir mi?
- Feature freshness gereksinimi nedir?
- Label ne kadar gec geliyor?
- Yanlis pozitif ve yanlis negatif maliyetleri nedir?
- Model outputu otomatik karar mi?
- Availability hedefi nedir?
- Hassas veri var mi?
- Global mi bolgesel mi deploy edilecek?

20.3 Trade-off cumle kaliplari

```
"Bu featurei request-time hesaplamak daha fresh sonuc verir; ancak latencyyi artirir.
Bu nedenle 5 dakikalik freshness yeterliyse stream ile precompute edip online storeda tutarim."

"Daha buyuk model offline metricte daha iyi olabilir; fakat p99 latency SLOsunu bozuyorsa
daha kucuk modeli production adayi olarak tercih ederim."
```

20.4 Sik yapilan hatalar

- Daha gereksinimleri sormadan teknoloji isimleri saymak.
- Model algoritmasina fazla zaman ayirip data/serving/monitoringi atlamak.
- Sadece average latency konusmak; p95/p99 belirtmemek.
- Training-serving skew ve leakage riskini gormemek.
- Failure/fallback tasarlamamak.
- Label delayi unutmak.
- Maliyeti ve business metricleri yok saymak.

21. Alistirmalar ve Tasarim Sorulari

21.1 Mini tasarim alistirmalari

59. Gunluk 5 milyon musteriyi churn riskine gore skorlayan sistem tasarla. Neden online serving gerekmeyebilir?
60. E-ticaret ana sayfasi icin recommendation sistemi tasarla. Candidate generation ve ranking katmanlarini ayir.
61. Ride-sharing ETA modelini tasarla. Hangi featurelar request-time, hangileri precompute olmali?
62. Bankacilik fraud modelinde online feature store cokerse nasil graceful degradation tasarlarsin?
63. Real-time modelin p99 latency degeri deploymenttan sonra 60 msden 220 mse cikti. Debug sirani yaz.
64. Model A AUC 0.91 ve p99 180 ms; Model B AUC 0.895 ve p99 35 ms. SLO p99<80 ms ise hangi trade-offlari tartisirsin?
65. Bir modelin input distributioni degisti ama business KPI sabit. Otomatik retrain eder misin? Hangi ek sinyalleri istersin?
66. Label 14 gun gec geliyorsa deployment sonrasi ilk 24 saatte hangi proxy metrikleri izlersin?

21.2 Mülakat seviyesi buyuk sorular

- YouTube/Netflix benzeri video recommendation sistemi tasarla.
- Uber/Maps benzeri ETA prediction sistemi tasarla.
- Ads click-through-rate prediction sistemi tasarla.
- E-commerce search ranking sistemi tasarla.
- Spam / abuse detection sistemi tasarla.
- Demand forecasting platformu tasarla.
- LLM destekli semantic search/RAG serving sistemi tasarla.

21.3 Uygulamali proje odevi

Kendi portfolio projen icin bir “ML System Design Document” yaz. Kod yazmadan once asagidaki basliklari doldur:

- Problem ve business KPI
- Functional requirements
- Non-functional requirements
- Traffic/data estimates
- High-level architecture
- Feature definitions + freshness
- Training pipeline
- Serving API
- Deployment strategy
- Monitoring dashboard
- Failure modes
- Retraining policy
- Security/privacy
- Cost assumptions

22. Terimler Sozlugu ve Tekrar Kontrol Listesi

Terim	Kisa aciklama
SLO	Servis icin hedeflenen olculebilir performans seviyesi.
p95/p99 latency	Isteklerin %95/%99unun altinda kaldigi gecikme.
Throughput	Birim zamanda islenen request/prediction sayisi.

Feature freshness	Feature degerinin ne kadar guncel oldugu.
Training-serving skew	Training ve serving feature hesaplarinin farkliligi.
Feature store	Offline/online feature yonetimi ve erisim katmani.
Model registry	Model artifact, version ve lifecycle metadata deposu.
Canary	Yeni surume kucuk trafik yuzdesi verme.
Shadow	Aday modeli gercek trafikte karar vermeden calistirma.
Drift	Data/label/concept dagilimlarinin zamanla degismesi.
Backfill	Gecmis tarihleri yeniden hesaplama.
Idempotency	Ayni islemin tekrarinin sonucu bozmamasi.
Circuit breaker	Hata veren dependencyye gecici istek kesme deseni.
Fallback	Ana prediction yolu calismazsa guvenli alternatif.
Point-in-time join	Her training ornegi icin sadece o anda mevcut featurelari kullanma.

22.1 Kendini test et

- [ ] Batch ve online inference arasindaki farki ve secim kriterini aciklayabiliyor muyum?
- [ ] Feature storeun hangi problemi cozdugunu biliyor muyum?
- [ ] Point-in-time correctness ve leakageyi ornekle aciklayabiliyor muyum?
- [ ] p95/p99 latency ve QPS kavramlarini sistem tasariminda kullanabiliyor muyum?
- [ ] Stateless serving ve horizontal scaling iliskisini kurabiliyor muyum?
- [ ] Canary, shadow, A/B ve blue/green farklarini biliyor muyum?
- [ ] Monitoringi infra-data-model-business olarak ayirabiliyor muyum?
- [ ] Data drift ile concept drifti ayirabiliyor muyum?
- [ ] Label gecikmesi varsa proxy metric dusunebiliyor muyum?
- [ ] Failure durumda timeout/retry/fallback/circuit breaker tasarlayabiliyor muyum?
- [ ] Bir predictionin model/data/feature versiyonunu izleyebilir miyim?
- [ ] Mimari kararlarin accuracy-latency-cost trade-offunu aciklayabiliyor muyum?

Modulun ozeti

ML System Design, “hangi algoritma?” sorusundan cok “bu model guvenilir bir urun sistemi icinde nasil yasayacak?” sorusudur. Senior Data Scientist ve ML Engineer seviyesinde fark yaratan yetkinlik; data, model, software ve infrastructure kararlarini ayni mimari uzerinde birlestirebilmektir.

Onerilen sonraki uygulama

Bu modulu tamamladiktan sonra en iyi pekistirme, mevcut portfolio projelerinden birini secip onun production mimarisini cizmektir. Ornegin Customer Churn icin batch scoring; Fraud Detection icin online serving; Recommendation icin candidate + ranking mimarisi tasarlanabilir.

Kaynak onerileri

- Designing Machine Learning Systems - Chip Huyen
- Designing Data-Intensive Applications - Martin Kleppmann
- Machine Learning Design Patterns - Valliappa Lakshmanan, Sara Robinson, Michael Munn
- Google Rules of Machine Learning
- MLflow, Feast, Kubernetes ve bulut saglayicilarinin production ML dokumantasyonlari